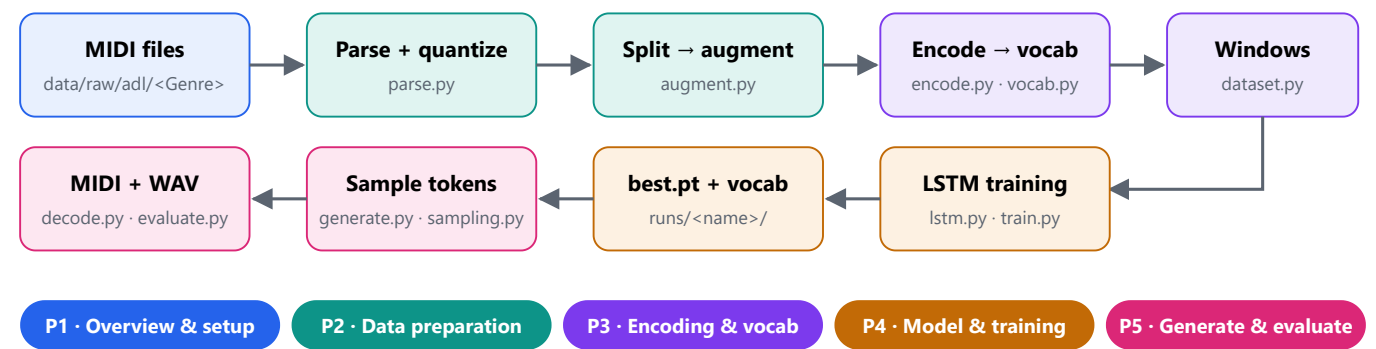


Symbolic Music Generation with LSTMs

How the code works, stage by stage

This guide explains the project folder `DL/` from the first MIDI file read to the last WAV written. It is split into **five parts, one per presenter**. Each part covers one stage of the pipeline, names the files involved, and ends with a short line you can say out loud.



The whole project is one idea: **predict the next musical symbol**, the way a text model predicts the next word.

Text model:	"the" → "cat" → "sat" → ?	word / character	→	a note, chord or rest
This project:	60.64.67 → <REST:2> → 62 → ?	sentence	→	a piece of music (MIDI)

Figure 1. The pipeline, colour-coded by presenter. Top row prepares data; bottom row trains, generates and plays it back.

How to split this guide between five people

Presenter	Topic	Pages	Files they should open and understand
Person 1	Overview, setup, configs, folder map	2	main.py, src/config.py, configs/*.yaml
Person 2	Reading MIDI, quantizing, split & augment	3–4	src/data/parse.py, augment.py, types.py
Person 3	Turning music into tokens, vocabulary, windows	5–6	src/data/encode.py, vocab.py, dataset.py
Person 4	The LSTM model and the training loop	7–8	src/models/lstm.py, src/train.py
Person 5	Generating music, audio, metrics, results	9–10	src/generate.py, sampling.py, decode.py, evaluate.py

Page 11 lists the corrections the slide deck needs. Page 12 is viva preparation that everyone should read. All numbers in this guide come from the files in `runs/`, `data/processed/` and `configs/`, not from memory.

Your job is to give the audience the map before anyone dives into details: what the project does, where each piece lives, and how one command runs everything.

The folder structure

```
DL/
├─ main.py           ← one entry point for everything
├─ configs/          ← one YAML file per experiment
│   ├── default.yaml (every setting, commented)
│   ├── adl_mixed.yaml (THE main run, "v5")
│   └── smoke.yaml   (tiny demo run)
├─ src/
│   ├── config.py    load YAML, overrides, hashing
│   ├── data/        parse → augment → encode
│   │               → vocab → dataset
│   ├── models/      lstm.py, transformer.py
│   ├── train.py     training loop
│   ├── generate.py   sampling loop
│   ├── sampling.py   temperature / top-k / top-p
│   ├── decode.py     notes → MIDI → WAV
│   └── evaluate.py   music metrics
├─ scripts/          one script per stage
├─ app/transcribe.py optional MP3 → MIDI seed
├─ tests/             383 automated tests
├─ data/             raw/ processed/ generated/
└─ runs/             one folder per trained model
```

Running it

`main.py` chains the stages. Each stage can also run alone.

Command	What it does
<code>main.py demo</code>	Synthetic data, full pipeline, a few seconds. Proves the install works.
<code>main.py prepare</code>	Raw MIDI → cached token arrays
<code>main.py train</code>	Train the model in the config
<code>main.py generate</code>	Write new <code>.mid</code> files
<code>main.py evaluate</code>	Score generated music
<code>main.py</code> (no command)	prepare + train + generate

Any setting can be changed without editing a file:

```
main.py train --config configs/adl_mixed.yaml
--set train.lr=0.0005
```

Configs: an experiment is a YAML file

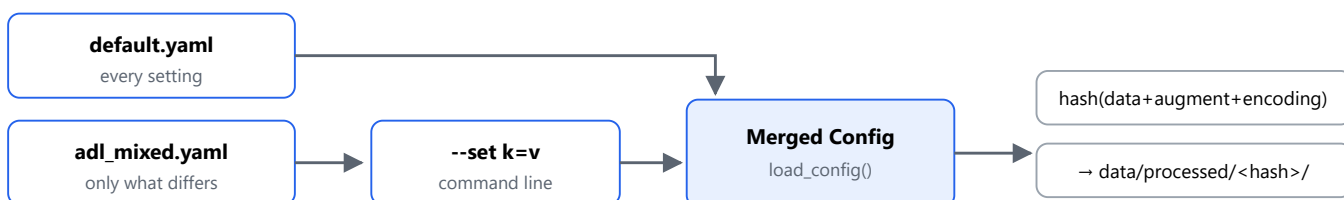


Figure 2. Configs layer on top of `default.yaml`. The data settings are hashed, so changing the grid or encoding creates a new cache instead of reusing a stale one.

Two "seams" make experiments cheap

Seam 1, encoding (`encoding.scheme`): `note_chord`, `interval`, `pitch_duration`, `event`.

Seam 2, architecture (`model.arch`): `lstm` or `transformer`.

Every other module works with any combination, so comparing them is a one-line config change.

The model went through five versions

v1 ran on a CPU (2×256 LSTM, 100-token context, ±3 keys). v2–v5 ran on an RTX 5070 GPU. v5 is the final model: 3×512 LSTM, 256-token context, ±6 keys, adaptive softmax. Training time fell from about 8 hours to 11 minutes.

"Our code is a pipeline of small modules. One YAML file describes an experiment, and `main.py` runs every stage of it in order."

A MIDI file is a list of notes with start and end times in seconds. Before a neural network can learn from it, we turn each file into a clean, regular **Piece**.

The two data types everything else uses (`types.py`)

NoteEvent

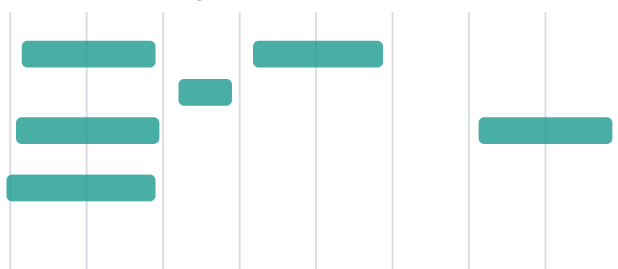
pitch (MIDI number, middle C = 60), start and duration **in beats**, velocity (loudness).

Piece

A list of NoteEvents sorted by time, plus `style` (Classical / Jazz / Blues, taken from the folder name) and `tempo`.

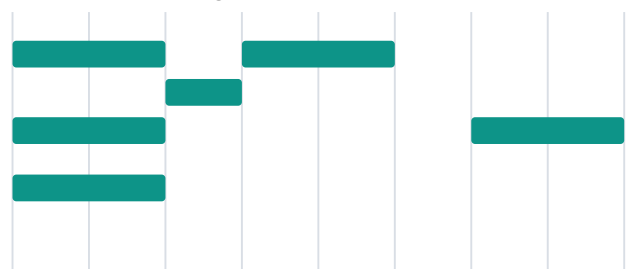
Quantization: snapping to a 16th-note grid

Before: human timing



start = 0.15, 0.08, -0.04 beats (played "together" by a human)

After: quantize(value, grid=0.25)



all three now start at exactly 0.00, so they form one chord

Figure 3. Each vertical line is one 16th note (0.25 beat). A human never plays exactly on the line; quantization rounds start and length to the nearest line.

What `_parse_file()` does to each file, in order

1. **Read** it with `pretty_midi` and check the time signature. A piece is kept only if every signature it uses is 4/4, 3/4 or 2/4.
2. **Collect notes** (`_collect_notes`). `track_strategy: merge` joins all non-drum tracks into one stream. That is safe because every ADL track is a piano.
3. **Convert seconds to beats**. With `tempo_map: true`, `_beat_mapper` follows every tempo change. 16% of files change tempo; using only the first tempo pushed 263 of them off the beat, the worst by more than 600 beats.
4. **Quantize** start and duration to the 0.25-beat grid with `quantize()`.
5. **Filter**: drop notes shorter than `min_duration` and pieces with fewer than `min_events` notes.
6. **Merge duplicate notes** (`_merge_duplicate_onsets`): the same pitch twice at one moment used to create fake chords like `60.60`.
7. **Label the style** from the folder name (`_style_from_path`).

`load_corpus()` runs this across all files in parallel worker processes.

The numbers at this stage

The ADL Piano MIDI dataset has 11,073 files in 18 genre folders. The three chosen genres hold 2,019 files (Classical 1,398, Jazz 492, Blues 129). After the time-signature and length filters, **1,654 pieces** (1.88 million notes) remain.

"Parsing turns messy human performances into tidy notes on a 16th-note grid, measured in beats rather than seconds."

This page covers the most important correctness rule in the project: **split first, augment second**.

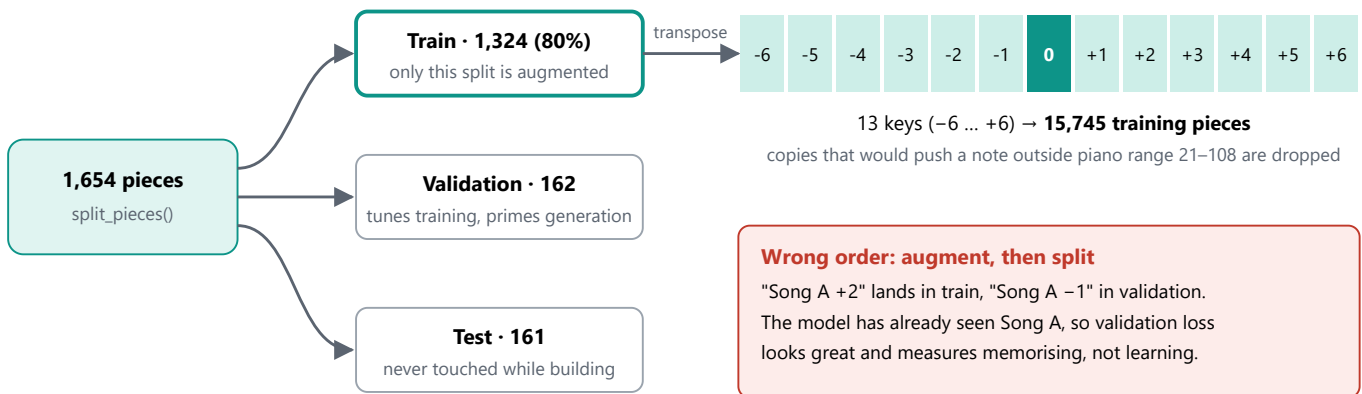


Figure 4. Splitting by whole piece before transposing keeps every version of a song on one side of the split.

`split_pieces(pieces, cfg)`

- Shuffles piece indices with a fixed seed (`split_seed: 1337`), so every run gets the same split and a checkpoint can always be re-evaluated on the exact data it was tested on.
- Takes 10% for validation, 10% for test, and the rest for training.
- **Removes duplicates:** ADL contains some pieces twice under different file names. Any validation or test piece whose notes match a training piece, *even in another key*, is dropped. That is why the final split is 1,324 / 162 / 161 rather than 1,324 / 165 / 165.

`transpose_augment(pieces, cfg)`

- Keeps every original and adds a copy shifted by each offset in `transpose_range` (v1 used ± 3 , so 7 keys; v3 onward use ± 6 , so 13 keys).
- A copy is dropped *whole* if any note would leave the piano range 21–108. Clamping single notes instead would distort the melody's shape.
- **Why transpose at all?** A melody means the same thing in any key. Showing it in all 12 keys teaches the model musical patterns, not "this tune is in C".

Honest result worth mentioning

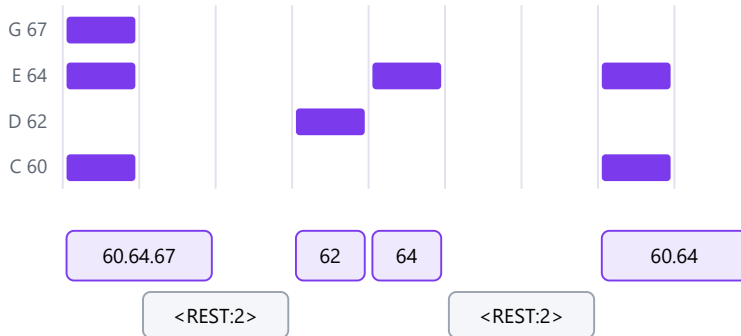
Going from ± 3 to ± 6 did **not** measurably improve validation loss (3.710 vs 3.714). It was kept because it covers every key, but it is a negative result, not a win.

"We split by piece before augmenting and we remove duplicate songs, so our validation score measures real generalisation."

A neural network cannot read notes; it reads integers. The encoder turns a Piece into a list of text **symbols**, and the vocabulary later turns each symbol into a number. This is "seam 1": the encoding scheme can be swapped in the config.

The `note_chord` encoder used by the final model

Piano roll (one column = one 16th-note step)



Rules

- notes starting together → one chord symbol, pitches joined by dots
- every note/chord symbol = one step
- silence = one run-length rest:
`<REST:3>` = 3 silent steps (max 16)
- `<STYLE:Jazz>` at the start of a piece tells the model its genre
- no durations in the symbol

Figure 5. Encoding a short phrase. Output: `<STYLE:Classical> 60.64.67 <REST:2> 62 64 <REST:2> 60.64 ...`

Two decisions that shaped the vocabulary

Decision	Why
Run-length rests <code><REST:k></code>	With one <code><REST></code> per silent step, 56% of all tokens were rests, and 75% of <i>generated</i> tokens. The model spent its capacity predicting silence. Run-length rests cut that to 36% and made sequences 26% shorter.
No duration in the symbol <code>include_duration: false</code>	Writing the length into the symbol (<code>60.64.67_1.000</code>) multiplies every chord by every possible length. On the same data the vocabulary was 363,455 symbols with durations and 17,886 without. Note length is rebuilt at playback time instead (Person 5).

The other encoders (seam 1)

Scheme	Symbols look like	Vocabulary	Result
note_chord (used)	<code>60</code> , <code>60.64.67</code>	37,020	Final model. In-key 0.884
interval	<code>+2</code> , <code>-5</code>	~400	"Differential music". Drifts out of key
pitch_duration	<code>P60</code> , <code>D1.000</code>	small	Melody only, no chords
event	<code>T2</code> <code>N60</code> <code>D4</code>	184	Exact durations, no <code><UNK></code> , but in-key fell to 0.61

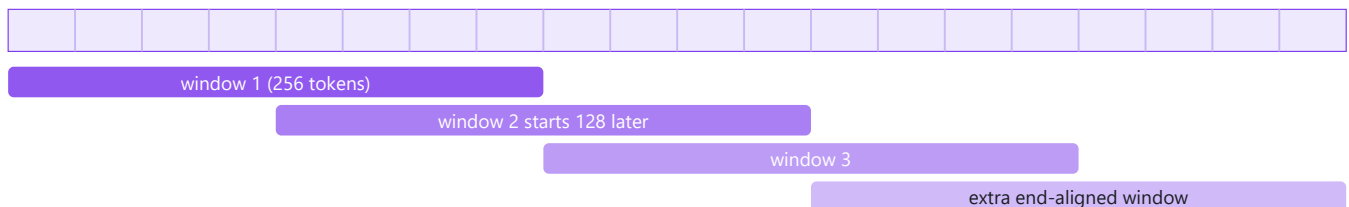
Every encoder has a `decode()` that reverses `encode()`. Tests check that `decode(encode(piece))` gives back the same music.

The vocabulary (`Vocab.build`)

- Maps each symbol to an integer id. Reserved ids come first: `<PAD>=0`, `<UNK>=1`, `<BOS>=2`, `<EOS>=3`, `<REST>=4`.
- Built from the **training split only**. Including validation or test symbols would leak information about them.
- Ids are assigned in `sorted()` order. Python set order changes between runs, and a reloaded model would then decode every token as the wrong note.
- **Pruning**: `min_freq: 10`. 60% of raw symbols appear only once and can never be learned, yet each costs an embedding row and an output column. Symbols seen fewer than 10 times are mapped to `<UNK>`; that affects about 4% of tokens. Rest and style tokens are never pruned, because losing a rest shifts every later note in time.
- Saved as `vocab.json` next to the trained model, so the model can never be paired with the wrong vocabulary.

Cutting sequences into training windows (`MusicDataset`)

One piece's token ids (each cell = one token)



stride 128 = half overlap, so each token is seen with a short and a long history · windows never cross into the next piece

Teacher forcing: y is x shifted one step left

x	<STYLE>	60.64.67	<REST:2>	62	...	y	60.64.67	<REST:2>	62	64
---	---------	----------	----------	----	-----	---	----------	----------	----	----

Figure 6. Each window of 256 inputs gives 256 targets: at every position the model must predict the next token.

Why teacher forcing matters

Scoring only the last position of each window threw away 99% of the work the network had already done. Scoring every position raised the training targets per epoch from 623,000 to 10.3 million for the same compute.

During training the model is always fed the *true* previous tokens, not its own guesses.

The processed cache

`prepare_data.py` saves token ids as `.npy` files in `data/processed/<hash>/`, plus `vocab.json` and `meta.json`. Final cache: **12.8 million training tokens**, vocabulary **37,020**.

`make_data_loaders()` shuffles training windows each epoch and serves them in batches of 48.

"Every chord becomes a word in a 37,020-word vocabulary, and we train on 256-token windows where every position is a prediction target."

`MusicLSTM` takes a batch of token ids `[B, 256]` and returns a probability for every vocabulary symbol at every position `[B, 256, 37,020]`. Position t predicts token $t+1$.

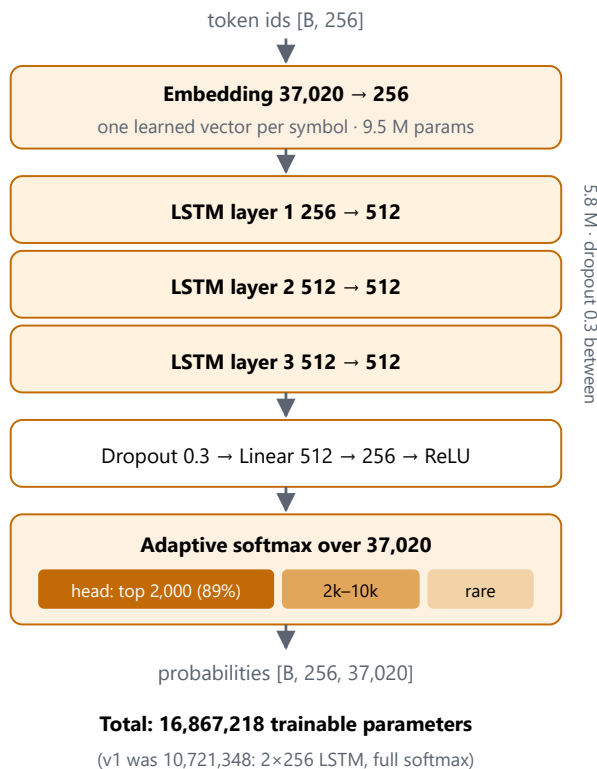


Figure 7. The final (v5) model.

Why an LSTM and not a plain RNN?

A plain RNN multiplies its memory by the same matrix at every step, so old information fades (vanishing gradients). An LSTM keeps a separate **cell state** c that is changed only by adding and gating, so a motif from many steps ago can still matter.

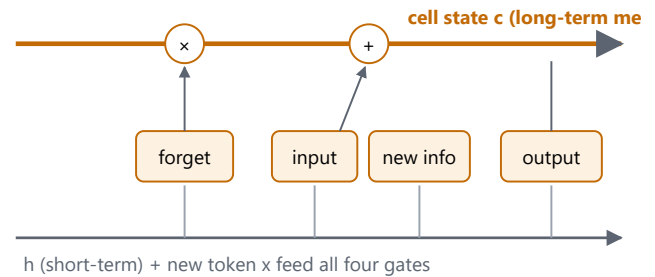


Figure 8. One LSTM cell: forget old memory, add new memory, decide what to output.

Details in `_init_weights()`

- **Orthogonal** recurrent weights, so repeated multiplication over 256 steps neither explodes nor vanishes.
- **Forget-gate bias = 1**, so the cell starts out remembering and learns when to forget.

Adaptive softmax (`model.output: adaptive`)

A normal output layer scores all 37,020 symbols at every position, which was the slowest and most memory-hungry part. Adaptive softmax scores the 2,000 most common symbols (89% of targets) at full size and the rare ones in smaller groups. The probabilities are still exact. Measured: **41% faster per batch**, GPU memory **5.9 GB → 1.2 GB**, and a better validation loss.

Seam 2: `transformer.py` has the same interface (8-head causal self-attention). It is built and tested on the demo data only; it has not been trained on the full corpus.

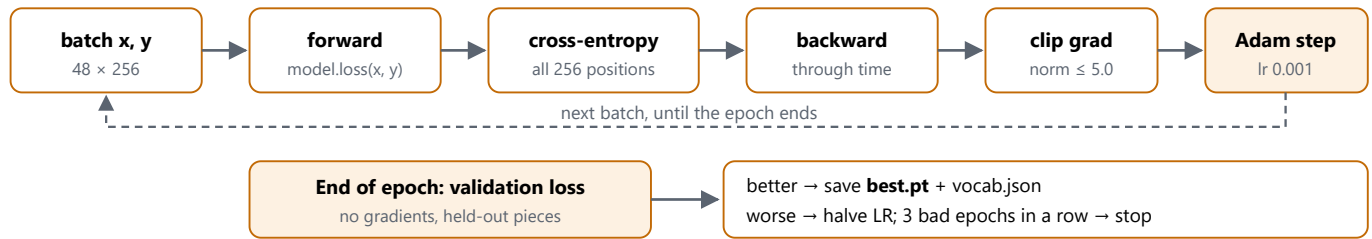


Figure 9. One training step (top) and the end-of-epoch decision (bottom), from `_run_epoch()` and `train()`.

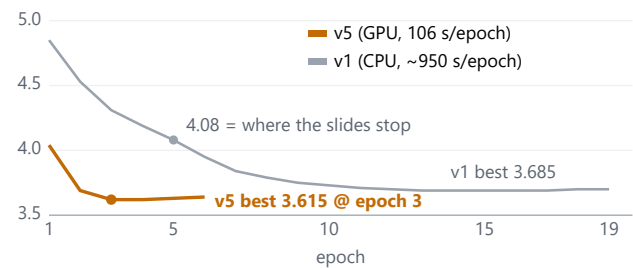
Key settings (v5)

Setting	Value
Loss	Cross-entropy, <PAD> ignored
Optimizer	Adam, learning rate 0.001
Gradient clipping	5.0
Batch / context	48 windows × 256 tokens
LR schedule	<code>plateau</code> : halve LR when val stops improving
Early stopping	<code>patience</code> : 3
Hardware	RTX 5070 GPU, ~106 s per epoch

Safety features

- **Atomic checkpoints:** written to a temp file, then renamed, so a crash cannot corrupt `best.pt`.
- **Resume** (`train.resume: true`) restores weights and Adam's state.
- **Config hash check:** a checkpoint refuses to load against data built with different settings.
- **VRAM warning:** on Windows a full GPU silently spills to system RAM and epochs become 20× slower.
- `history.csv` logs every epoch.

Validation loss by epoch (real data)



From `runs/ad1_mixed_lstm*/history.csv`. v5 overfits after epoch 3: training loss keeps falling (2.86) while validation rises, so early stopping ends the run at epoch 6 and keeps the epoch-3 weights.

Tried and rejected: bf16 mixed precision

25% faster, but validation loss after one epoch was 5.35 vs 4.80 in normal precision. Speed alone was not a good enough reason.

"We train with Adam and cross-entropy, keep only the best checkpoint by validation loss, and stop as soon as the model starts to memorise."

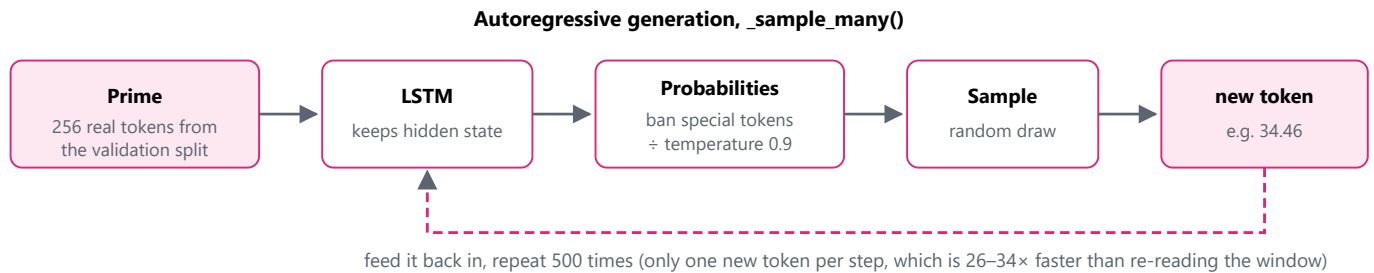


Figure 10. The model writes one token, reads it back, and writes the next.

Step by step (`generate()`)

1. Load `best.pt` and the `vocab.json` beside it. Refuse to run if the vocabulary does not match the data it was trained on.
2. **Prime** with a real excerpt from the **validation** split. The model never trained on it, so it cannot simply recite a memorised piece. You can also prime from your own MIDI (`--seed-midi`) or an MP3 via `app/transcribe.py`.
3. **Sample** the next token (`sampling.py`).
4. Repeat. Write the tokens as `.txt`, decode them to notes, write `.mid`.

Each run prints its random seed, so any sample can be reproduced with `--seed N`.

From tokens to sound (`decode.py`)

Decoded: every note one 16th long



sounds like staccato blips

`fill_durations()`: ring until the next onset (max 4 beats)



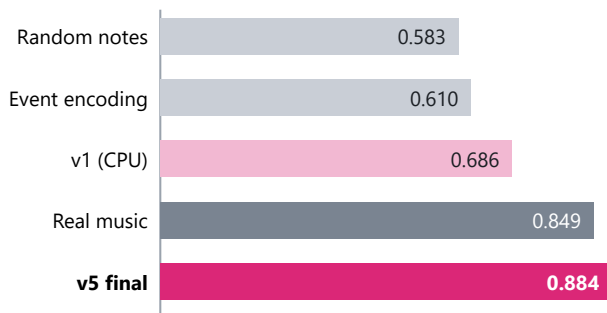
80% of real note lengths within half a beat (vs 66% before)

The `note_chord` model never predicts note length, so `fill_durations()` rebuilds it at playback time. **This is what fixed the "random, staccato" sound**, not the vocabulary change. `render_audio.py` then synthesises a 44.1 kHz stereo WAV with a pure-numpy piano model (decaying partials, reverb, loudness normalisation). The instrument is chosen at render time; the model only decides *which* notes, not how they sound.

Loss tells us how well the model predicts held-out music. It does not tell us whether the output *sounds* musical, so `evaluate.py` also scores the generated MIDI directly.

Metric	What it measures	Real music	v5 model
in_key_pct	Fraction of notes inside the estimated key (Krumhansl–Schmuckler key finder). Random notes score 0.583.	0.84–0.88	0.884
perplexity	$\exp(\text{loss})$: roughly how many symbols the model is choosing between. Lower is better.	—	37 of 37,020
distinct_n	Variety of note n-grams. Catches a model that repeats one safe phrase.	0.61	0.74
rhythm_div	Variety of gaps between notes	0.394	0.363
repetition_rate	Same note immediately repeated	0.028	0.055
copy_metrics	Plagiarism check: overlap with training n-grams (lower than real unseen music)	—	max copy $\approx$ 11 tokens

In-key score



Bar length is proportional to the score. v1 0.686 and real 0.849 are from the T=0.9 tuning table in `configs/ad1_mixed.yaml1`; v5 from `data/generated/results.md`.

Every version, best validation loss

Run	What changed	Val loss	PPL
v1	CPU, 2×256, ctx 100, ± 3	3.685	39.9
v2	GPU, 3×512, ctx 256	3.710	40.8
v3	± 6 transposition	3.714	41.0
v4	Data bugs fixed, tempo map	3.682	39.7
v5	Adaptive softmax, LR schedule	3.615	37.2

v1–v3 used a 25k–38k vocabulary and v4–v5 37,020, so losses are close but not perfectly comparable. The event-encoding run (loss 1.68) uses a different vocabulary and cannot be compared at all.

The data-pipeline audit before v4

Three bugs changed what the model trained on: (1) `<STYLE>` tokens were silently dropped, so v1–v3 never saw a genre label; (2) the same pitch twice at one moment made fake chords like `60.60`; (3) duplicate songs leaked from train into validation and test. Piece endings were also never used as targets. All four are fixed and covered by `tests/test_data_audit.py`.

Known limitations

- **No long-range form.** 256 tokens $\approx$ 93 beats $\approx$ 23 bars of 4/4. The model cannot bring back a theme from a minute earlier. This is why a Transformer arm exists.
- **Overfits early** (after about 3–5 epochs). More varied data would help more than a bigger model.
- **Durations are estimated, not learned**, and about 4% of chords are `<UNK>`.
- **Weak style conditioning**: the style token is only at the start of a piece, so most windows never see it.

"The final model keeps 88% of its notes in key, the same level as real music, and it does not copy its training data."

The slides describe **v1**, the first CPU run. The code has moved on to **v5**. Either relabel the deck as "first experiment", or update it with the v5 numbers below. Present the newer results, because they are much stronger.

Slide	Slide says	What the code and logs show
5	Filtered to Classical/Jazz/Blues "leaving 1,654 high-quality pieces"	Those genres hold 2,019 files. 1,654 is what remains after the time-signature and length filters. The current split is 1,324 / 162 / 161 after duplicate removal; training data is ±6 → 15,745 pieces , not ±3 → 8,885 .
6	Style markers such as <code><STYLE:Jazz></code> were added	In the v1 run these tokens were silently dropped by a bug; v1's vocabulary contains zero style tokens. They exist only from v4 on.
7	Vocabulary 25,028; two LSTM layers; Linear + softmax; 10.7 M parameters	True for v1 only. v5: vocabulary 37,020 , 3 layers × 512 , Linear 512→256 + ReLU, then adaptive softmax , 16.9 M parameters. The slide also leaves out the hidden size, dropout 0.3 and the extra Linear+ReLU layer.
8	100-token sequences, batch 128, trained entirely on CPU	v1 only. v5: 256 tokens, batch 48, RTX 5070 GPU , 106 s/epoch, ~11 min total. Missing: gradient clipping 5.0, early stopping (patience 3), plateau LR schedule.
9	Removing durations + min-count 10 took 363,455 → 25,028	Two different datasets are mixed here. On the same data, removing durations gave 363,455 → 17,886 . 25,028 came after the time-signature filter was widened (more pieces). Rare symbols are mapped to <UNK> (~4% of tokens), not deleted.
10	Validation loss improved 4.85 → 4.08	That is only epochs 1–5 of v1. v1 reached 3.685 ; v5 reached 3.615 (perplexity 37).
10	In key 82.8% vs 88.2% for real music	These numbers are not in any results file. The recorded figures are v5 0.884 , real music 0.84–0.88, random 0.583. The model now matches real music.
10	"After the vocabulary fix, the model learned to hold notes"	Wrong cause. The model does not predict durations at all. Notes stopped sounding staccato because <code>fill_durations()</code> stretches each note to the next onset at playback time.
11	"The model only remembers 1–2 bars"	v1's 100 tokens cover about 9 bars; v5's 256 tokens cover about 23 bars (each token ≈ 0.36 beats with run-length rests). The limit is real but the number is wrong.
11	Future: use GPUs, reintroduce exact note lengths, try Transformers	GPU training is already done (8 h → 11 min). Exact durations were already tried with the event encoding (in-key fell to 0.61). The Transformer is implemented but not trained on the full data.

Missing from the deck entirely

- Split-before-augment and duplicate removal (data leakage)
- Run-length rests (`<REST:k>`) and why they were needed
- Tempo-map fix (16% of files change tempo)
- Adaptive softmax: 41% faster, 5.9 → 1.2 GB
- Sampling: temperature 0.9, no top-p, banned tokens, priming from validation
- The v1 → v5 progression and the data-pipeline audit
- Evaluation beyond in-key: distinct-n, rhythm, repetition, copy check
- Negative results: bf16, event encoding, ±6 transposition
- 383 automated tests
- Citations in the literature review (it names "Differential Music" but no paper)
- A results chart and an audio demo
- Presenter names on the title slide

What is teacher forcing?

In training the model is given the true previous tokens, not its own guesses. Every position in the window becomes a target, so one 256-token window gives 256 training examples.

Why sample instead of picking the most likely token?

The most likely continuation of a repeated note is usually that note again, so greedy decoding gets stuck repeating one note.

What does perplexity 37 mean?

On average the model is as unsure as if it were choosing between 37 equally likely symbols, out of 37,020. It has narrowed the field about a thousand-fold.

How do you know it isn't copying?

It is primed with validation music it never trained on, and `copy_metrics` shows the longest verbatim match is about 11 tokens, with less training-set overlap than real unseen music.

Why is the vocabulary so large?

Every distinct combination of simultaneous notes is its own symbol, and real piano music has tens of thousands of different chords.

What is adaptive softmax?

An output layer that scores the 2,000 common symbols in full and the rare ones in smaller groups. Same exact probabilities, 41% faster, a fifth of the memory.

Why is the split done before augmenting?

Otherwise transposed copies of a validation song end up in training, and validation measures memory, not generalisation.

Why an LSTM and not a plain RNN?

Gates and a separate cell state stop the memory from fading, so information from hundreds of steps back can still affect the next note.

What would you do with more time?

Train the Transformer on the full corpus (long-range form is the main weakness), and pair the event encoding with a stronger model so it keeps exact durations and stays in key.

Which metric matters most?

`in_key_pct`, read together with `distinct_n`: a model that repeats one safe phrase also scores high on key.

What surprised you?

Data preparation mattered more than the model. Run-length rests, duplicate removal and the tempo fix each changed results more than adding layers did.

Can someone reproduce your results?

Yes. The split seed is fixed, every generation prints its seed, the vocabulary is saved beside the model, and 383 tests check the pipeline.

Glossary

Token / symbol	One note, chord or rest	Epoch	One pass over the training data
Quantization	Snapping times to a musical grid	Checkpoint	Saved model weights (<code>best.pt</code>)
Transposition	Shifting a piece up or down in pitch	Early stopping	Stop when validation stops improving
Temperature	How adventurous sampling is	Seam	A swap point: encoding or architecture